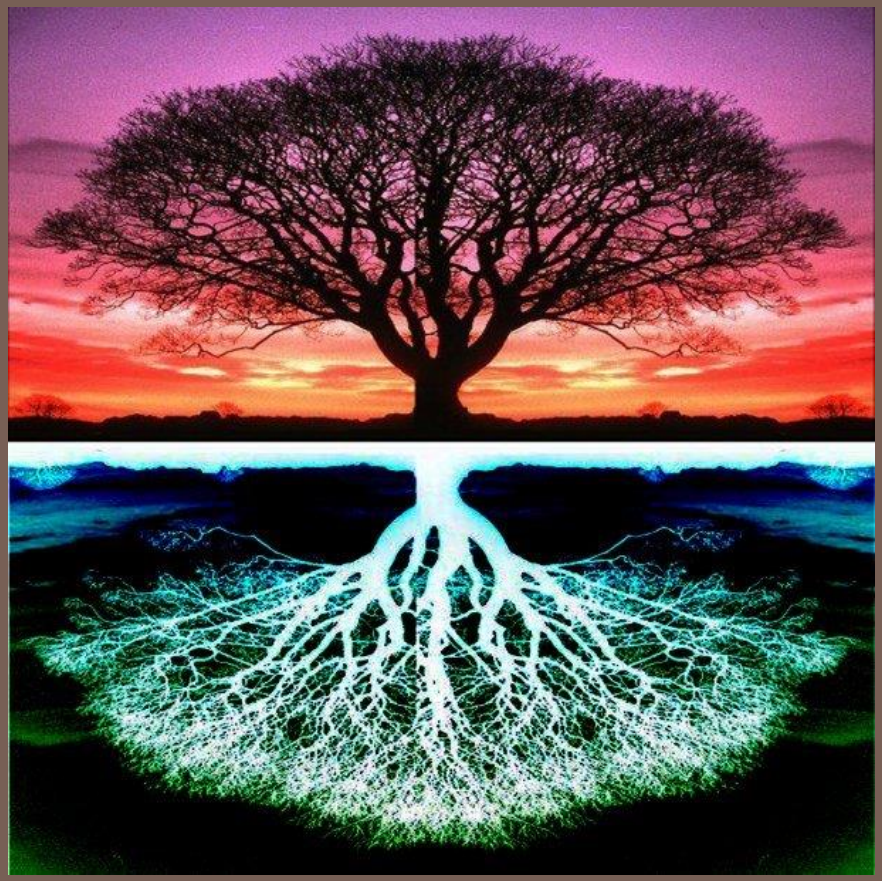


TREES

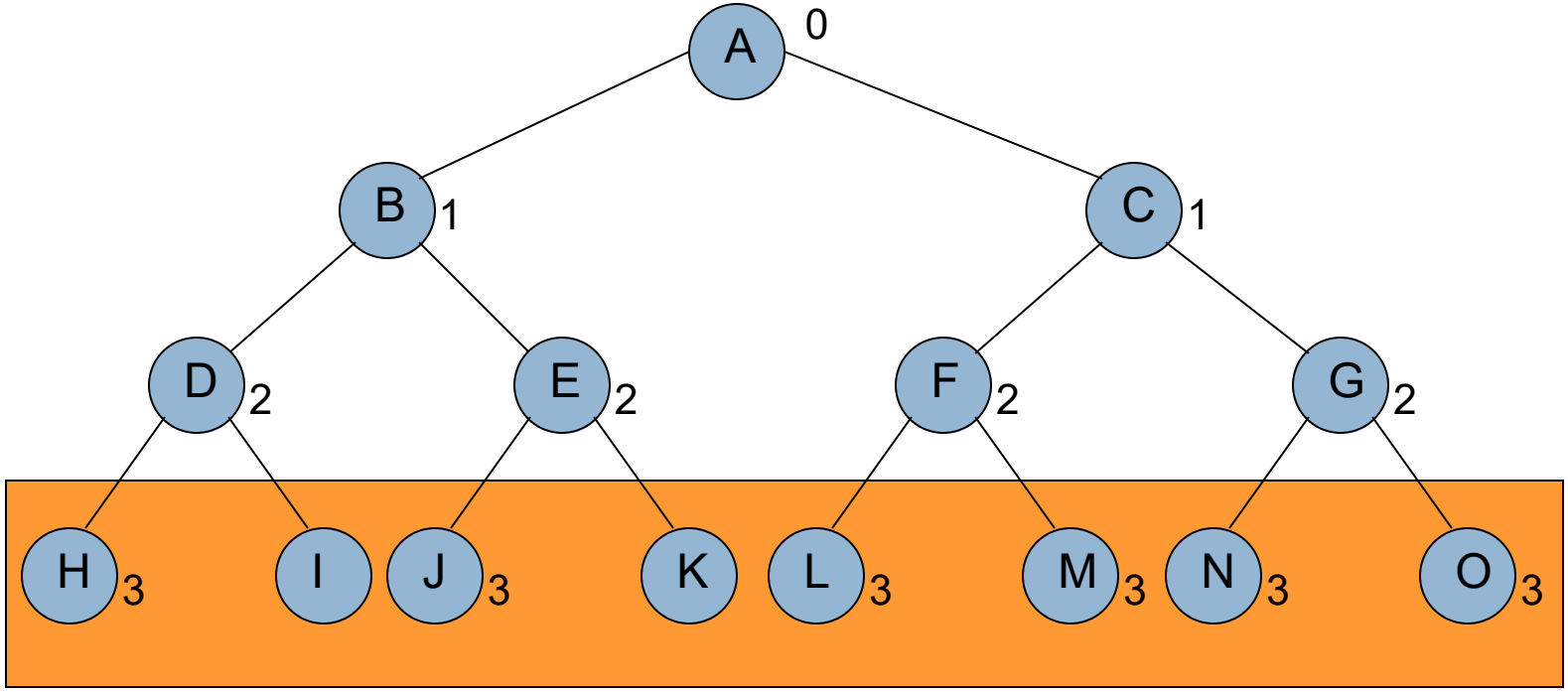
assadullahkhan

AssadUllahKhan

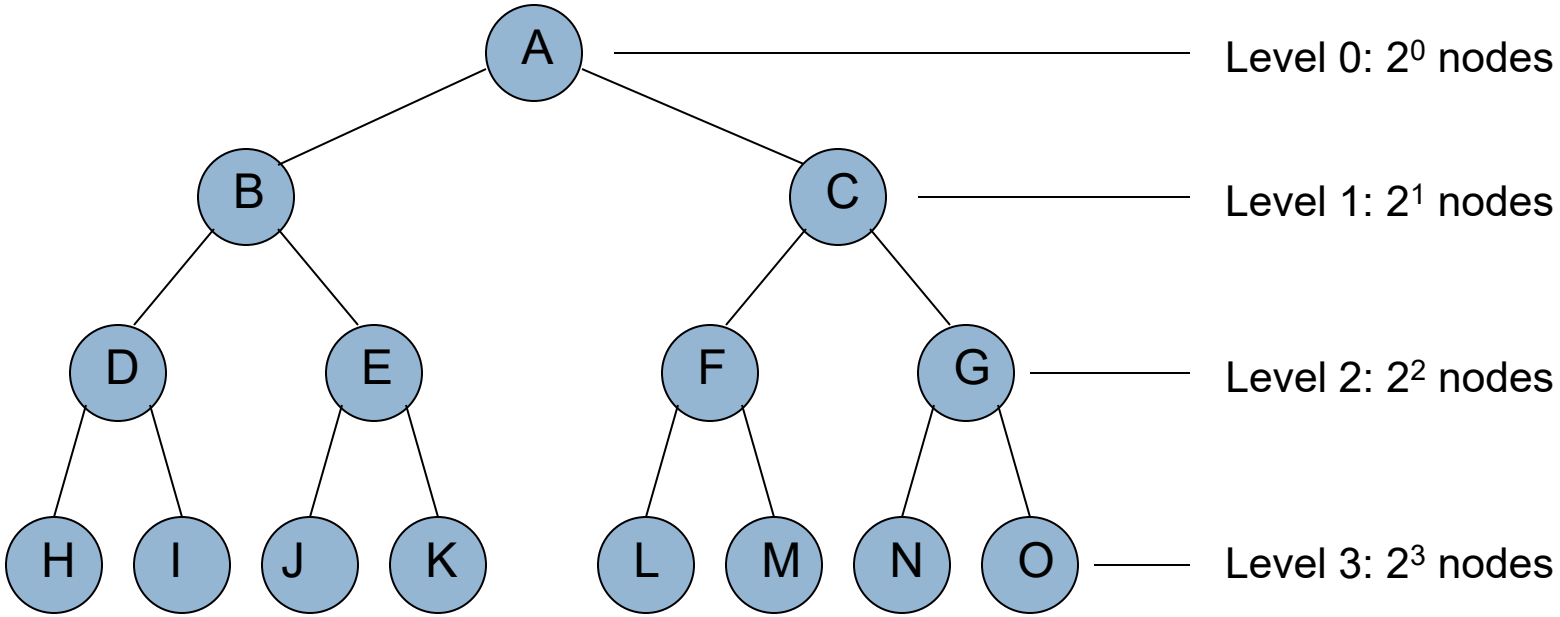
Research Assistant: **Mr. Assad Ullah Khan**

Complete Binary Tree

- A *complete binary tree* of depth *d* is the strictly binary all of whose leaves are at level *d*.

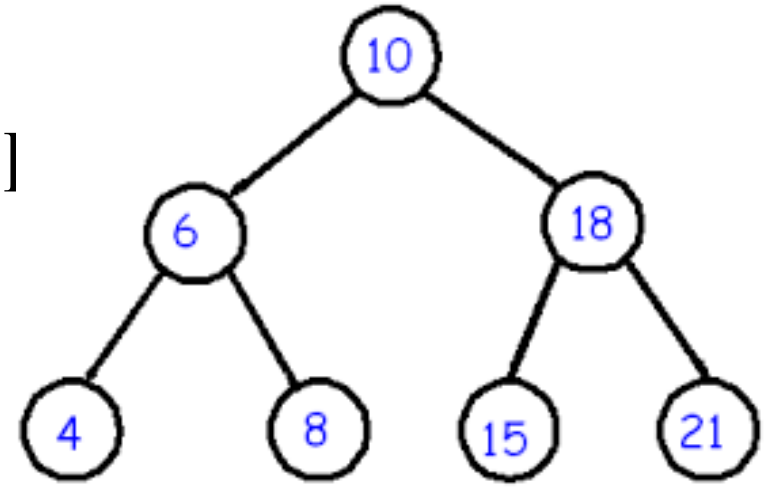


Complete Binary Tree



Vector Representation of Complete Binary Tree

- Tree navigation
  - Parent of **v[k] = v[(k – 1)/2]**
  - Left child of **v[k] = v[2*k + 1]**
  - Right child of **v[k] = v[2*k + 2]**



10	6	18	4	8	15	21
0	1	2	3	4	5	6

Binary versus General tree

General Tree

- Each node can have infinite number of children,
- **Can't** be empty.
- No limit on the **degree of node**
- Root have **in-degree 0** and maximum **out-degree n**
- Each **node** have in-degree **one** and maximum out-degree **n**.

Binary Tree

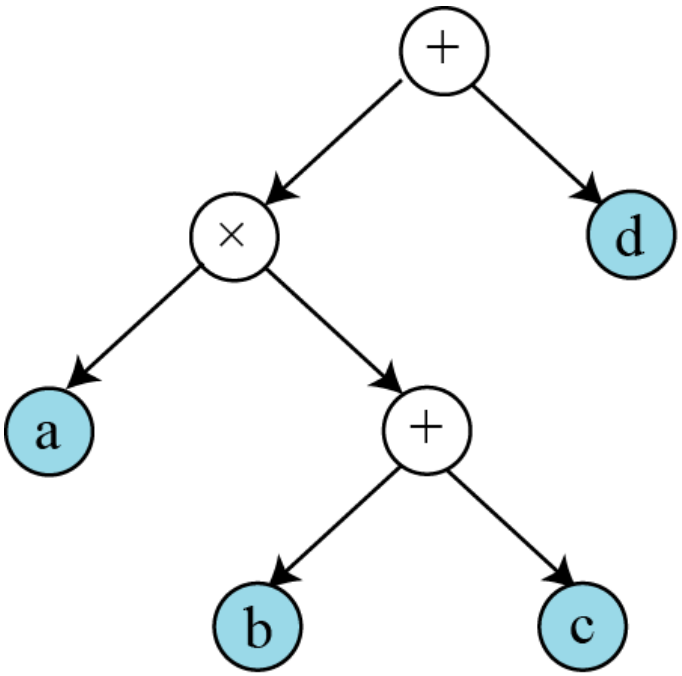
- Each node has at most **two** nodes
- **Can** be empty.
- It can't have more than **degree 2**
- Root has **in-degree 0** and maximum **out-degree 2.**
- Each node have in-degree **one** and maximum out-degree **2.**

Expression trees

- An **expression tree** is a binary tree with these properties:
  1. Each leaf is an operand.
  2. The root and internal nodes are operators.
  3. Subtrees are subexpressions with the root being an operator.
- An arithmetic expression can be represented in three different formats: infix, postfix and prefix.

Prefix: + A B Infix: A + B Postfix: A B+

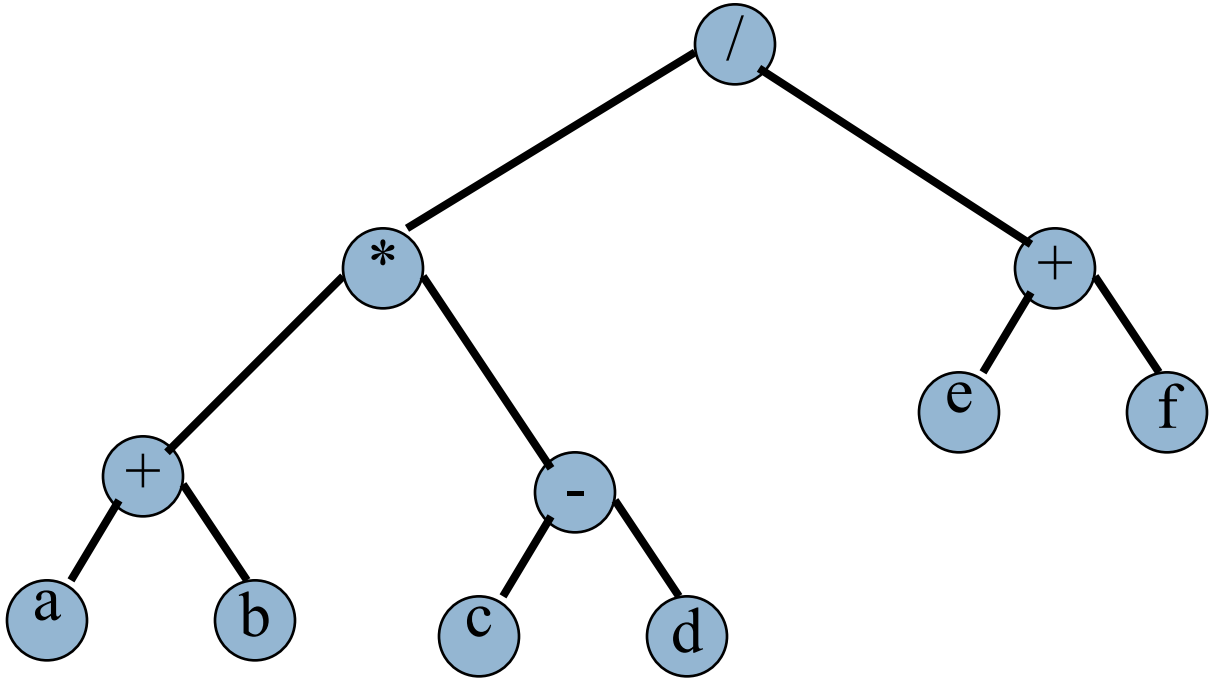
Expression trees



Expression tree

+ × a + b c d	Prefix notation
a × (b + c) + d	Infix notation
a b c + × d +	Postfix notation

Preorder Of Expression Tree



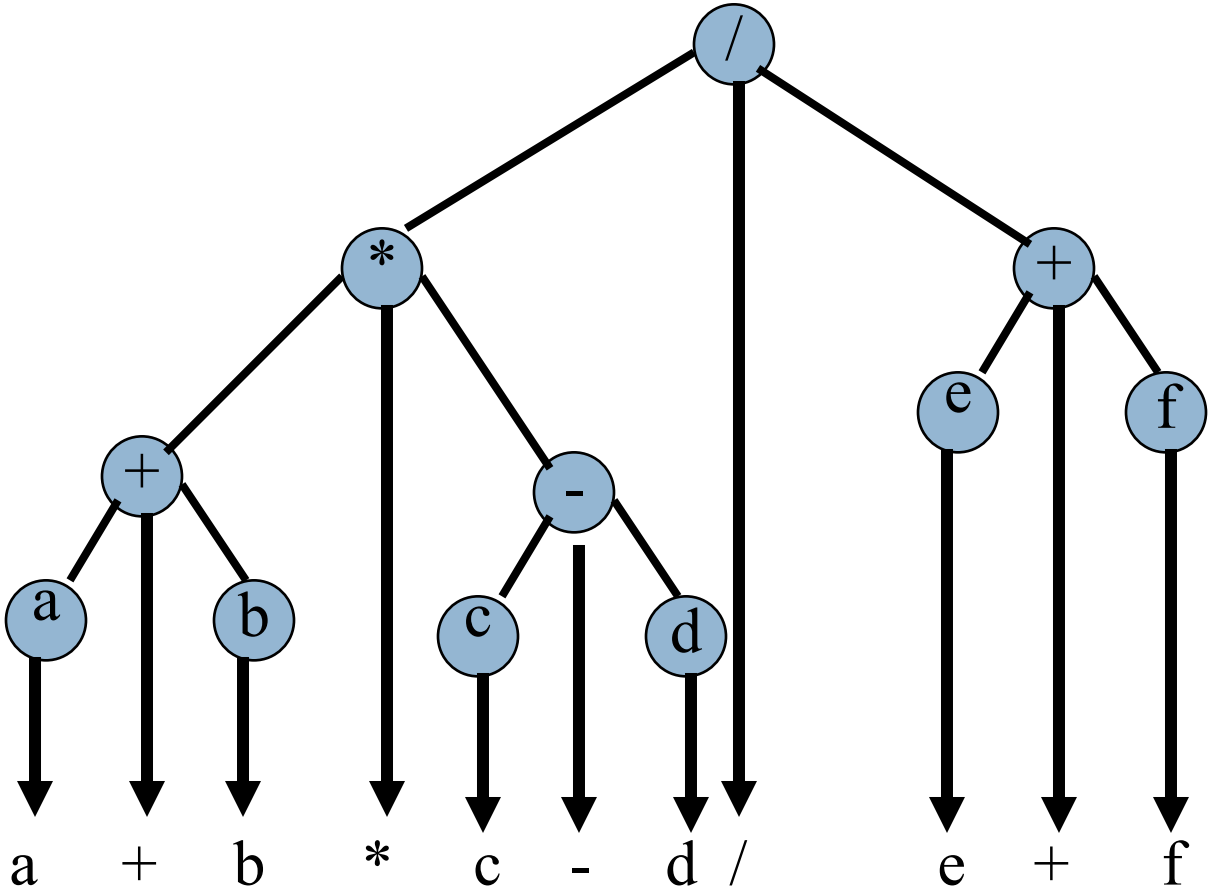
/ * + a b - c d + e f

Gives prefix form of expression!

Algorithm

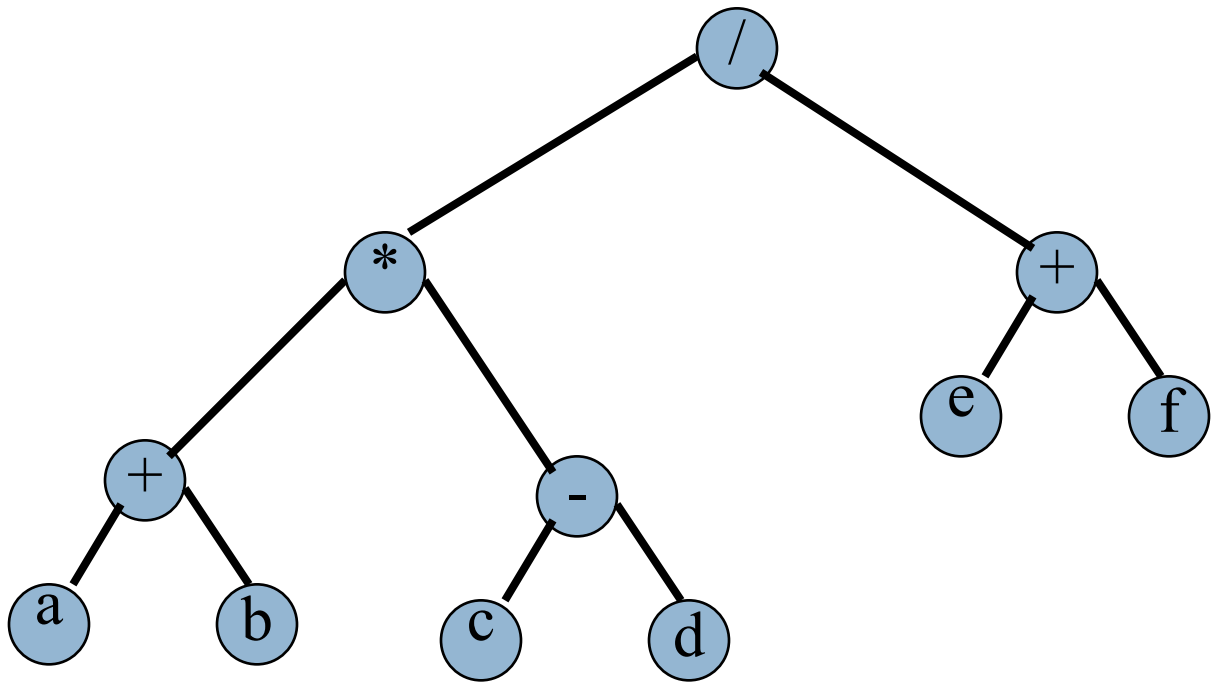
- Until all nodes are traversed −
  - **Step 1** − Visit root node.
  - **Step 2** − Recursively traverse left subtree.
  - **Step 3** − Recursively traverse right subtree.

Inorder Of Expression Tree



Gives infix form of expression (sans parentheses)!

Postorder Of Expression Tree



a b + c d - * e f + /

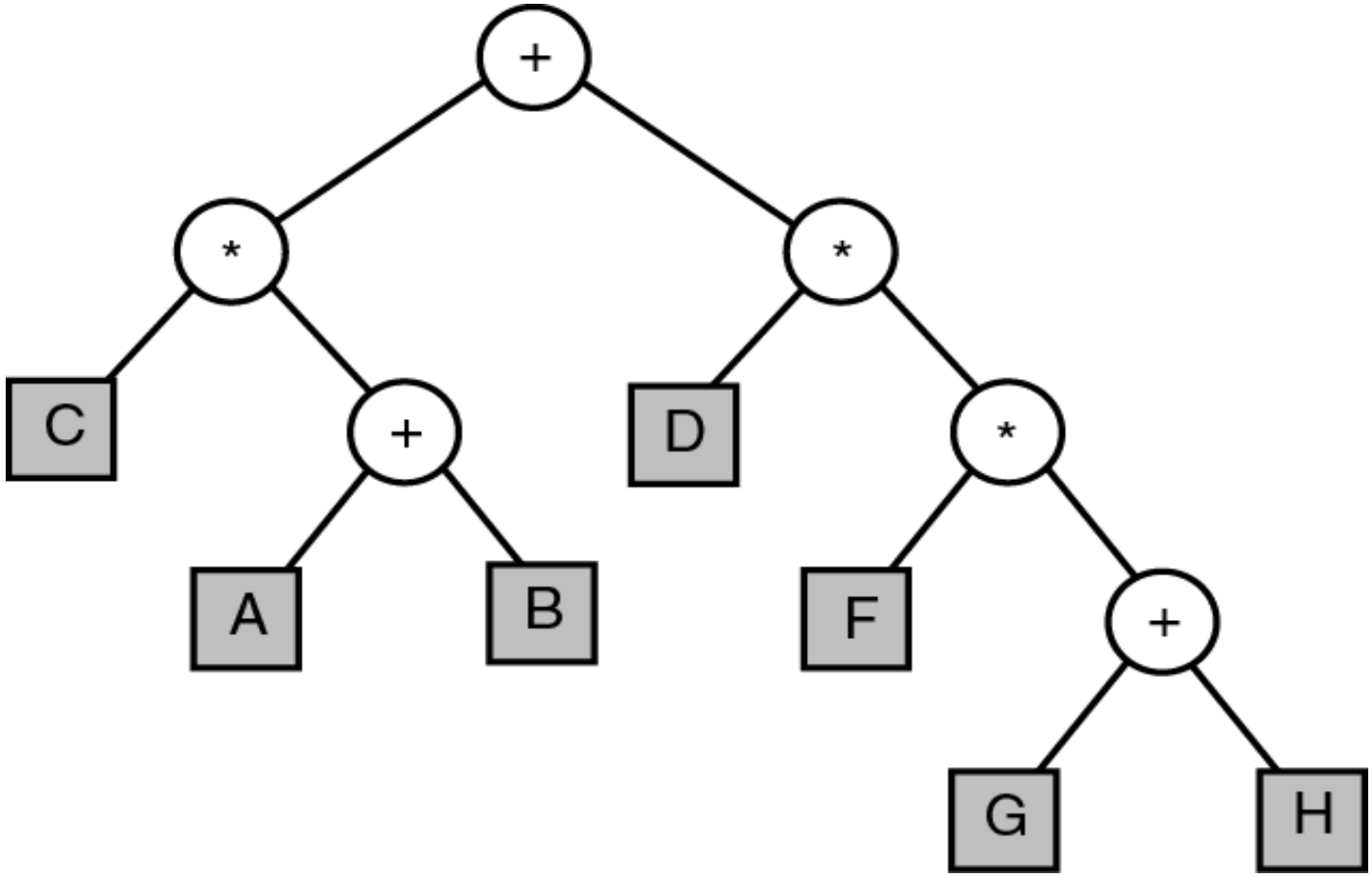
Gives postfix form of expression!

Algorithm

- Until all nodes are traversed –
  - **Step 1** – Recursively traverse left subtree.
  - **Step 2** – Recursively traverse right subtree.
  - **Step 3** – Visit root node.

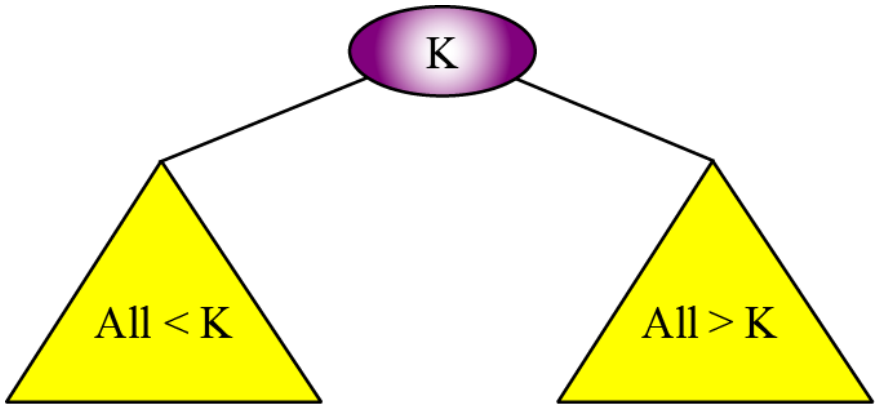
Exercise

Find the infix, prefix and postfix expressions of the below tree.



Binary Search Trees

A binary search tree (BST) is a binary tree with one extra property: the key value of each node is greater than the key values of all nodes in each left subtree and smaller than the value of all nodes in each right subtree.



Binary search tree (BST)

Binary Search Tree (BST)

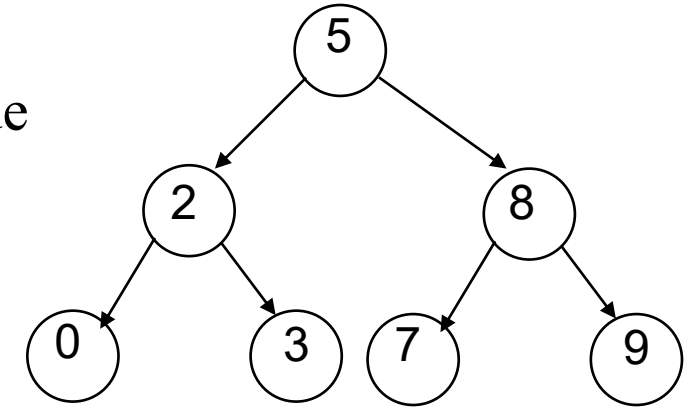
- Stores keys in the nodes in a way so that searching, insertion and deletion can be done efficiently.
- The smallest element in a binary search tree (BST) is the “left-most” node
- The largest element in a BST is the “right-most” node
- Inorder traversal of a BST encounters nodes in increasing order

If the tree data are *ordered and no duplicate values*:

in every subtree,

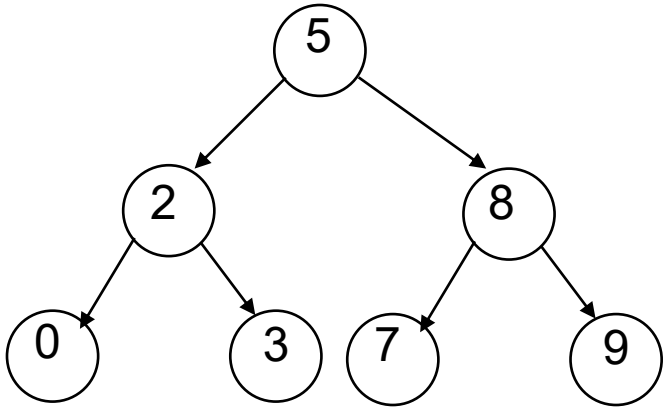
All *left* descendents of node come *before* node

All *right* descendents of node come *after* node



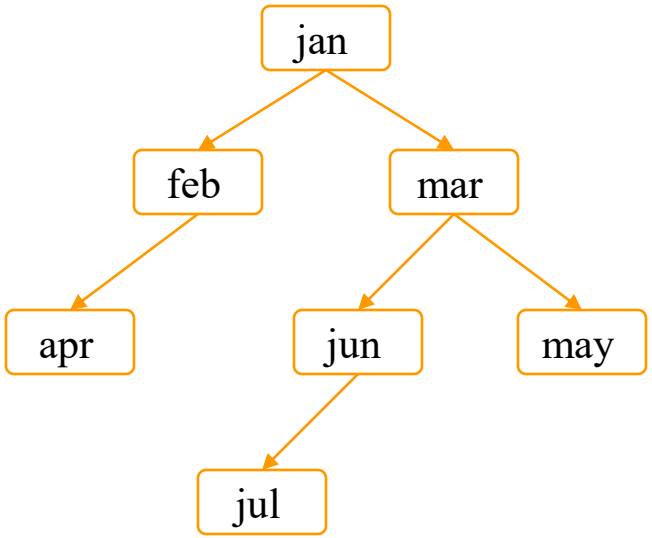
Binary Search using BST

- Assumes nodes are organized in a totally ordered binary tree
  - Begin at root node
  - Descend using comparison to make left/right decision
    - if (search_value < node_value) go to the left child
    - else if (search_value > node_value) go to the right child
    - else return true (success)
  - Until descending move is impossible
  - Return false (failure)
- *Search for 7 in this BST*



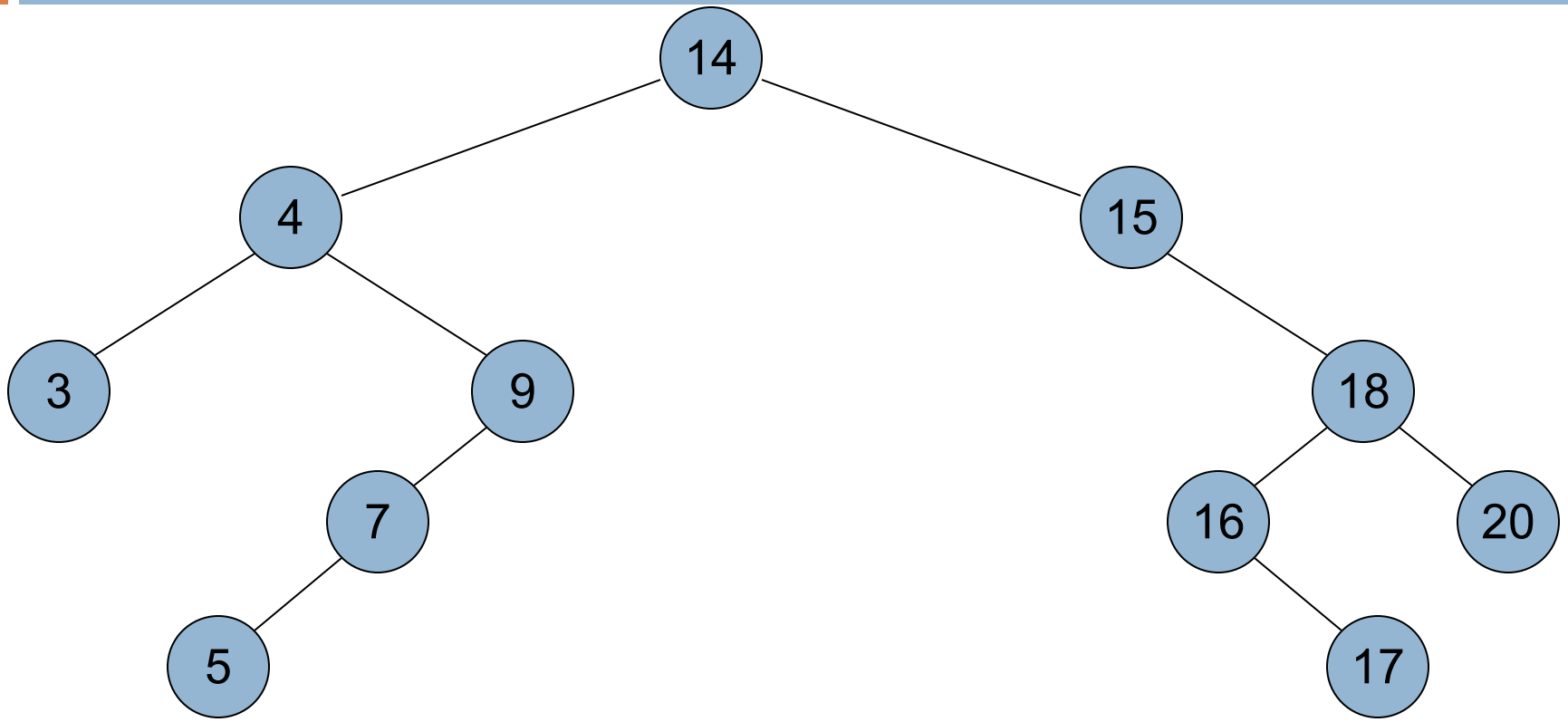
Building a BST

- To insert a new item
  - Pretend to look for the item
  - Put the new node in the place where you fall off the tree
- This can be done using either recursion or iteration
- Example
  - Tree uses *alphabetical order*
  - Months appear for insertion in *calendar order*



Build a BST 14 4 3 9 7 5 15 18 16 17 20

Building a BST



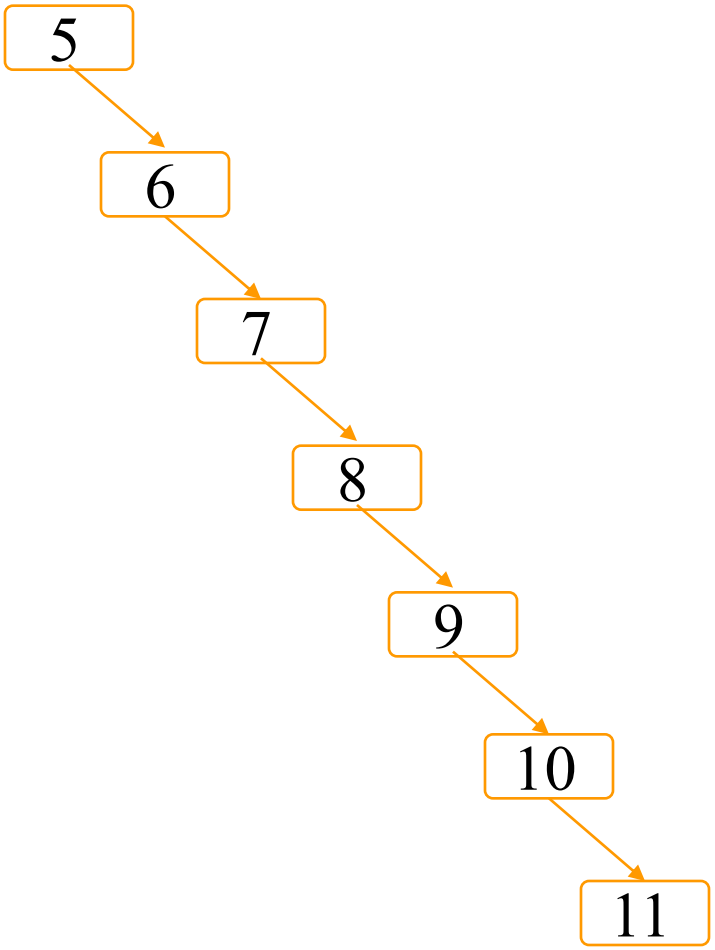
Build a BST 14 4 3 9 7 5 15 18 16 17 20

What can go wrong?

A BST makes searches very fast, *unless*…

- Nodes are inserted in increasing order
- In this case, we're basically building a linked list (with some extra wasted space for the **left** fields, which aren't being used)

BST works great if data arrives in random order



Binary versus Binary Search tree

Binary Tree

- It is a specialized form of tree with two child (left child and right Child).
- It is simply representation of data in Tree structure

Binary Search Tree

- It is a special type of Binary Tree that provides the efficient way of data sorting, searching and retrieving
- It follows following three conditions:

  1. Left child node is **smaller** than its parent Node

  2. Right child node is **greater** than its parent Node

  3. No duplicate values